

-- 1. Customers

```
CREATE TABLE Customers (  
  customer_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  email VARCHAR(100),  
  city VARCHAR(50),  
  signup_date DATE );
```

-- 2. Suppliers

```
CREATE TABLE Suppliers (  
  supplier_id INT AUTO_INCREMENT PRIMARY KEY,  
  supplier_name VARCHAR(100),  
  contact_email VARCHAR(100),  
  city VARCHAR(50) );
```

-- 3. Shippers

```
CREATE TABLE Shippers (  
  shipper_id INT AUTO_INCREMENT PRIMARY KEY,  
  shipper_name VARCHAR(100),  
  contact VARCHAR(100) );
```

-- 4. Payment Methods

```
CREATE TABLE Payment_Methods (  
  payment_id INT AUTO_INCREMENT PRIMARY KEY,  
  payment_type VARCHAR(50) UNIQUE );
```

-- 5. Products

```
CREATE TABLE Products (  
  product_id INT AUTO_INCREMENT PRIMARY KEY,  
  product_name VARCHAR(100),  
  category VARCHAR(50),  
  price DECIMAL(10,2),  
  stock_qty INT,  
  supplier_id INT,  
  FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id) );
```

-- 6. Orders (normalized, no total_amount column)

```
CREATE TABLE Orders (  
  order_id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT,  
  order_date DATE,  
  payment_id INT,  
  shipper_id INT,  
  FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
  FOREIGN KEY (payment_id) REFERENCES Payment_Methods(payment_id),
```

```
FOREIGN KEY (shipper_id) REFERENCES Shippers(shipper_id) );
```

```
-- 7. Order_Items
```

```
CREATE TABLE Order_Items (  
  order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT,  
  product_id INT,  
  quantity INT,  
  price_each DECIMAL(10,2),  
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES Products(product_id) );
```

```
select * from customers;  
select * from suppliers;  
select * from shippers;  
select * from payment_methods;  
select * from orders;  
select * from order_items;
```

```
#          -----|| Advance Assignment Questions ||-----
```

```
# a) Joins, Group By, Order By, Aggregations:
```

```
-- 1. Find the total revenue collected by each shipper.
```

```
Select shippers.shipper_name, sum(order_items.quantity * order_items.price_each) As  
total_revenue From Orders JOIN Order_Items ON Orders.order_id = Order_items.order_id  
JOIN Shippers ON orders.order_id = Shippers.shipper_id GROUP BY  
Shippers.shipper_name;
```

```
-- 2. Show the top 5 highest-spending customers along with their total payments.
```

```
Select c.name, sum(oi.quantity * oi.price_each) As total_spent From Customers c JOIN  
Orders o ON c.customer_id = o.customer_id JOIN Order_items oi ON o.order_id =  
oi.order_id GROUP BY c.name ORDER BY total_spent Desc LIMIT 5;
```

```
-- 3. Find product categories where the average selling price is greater than 8000.
```

```
Select category, avg(price) As avg_selling_price From Products GROUP BY category  
HAVING avg_selling_price > 8000;
```

```
-- 4. Show the total number of orders placed per city, sorted by highest to lowest.
```

```
Select c.city, count(order_id) As total_orders From Customers c JOIN Orders o ON  
c.customer_id = o.customer_id GROUP BY city ORDER BY total_orders Desc;
```

```
-- 5. Find suppliers who supply more than 1 product category.
```

```
Select s.supplier_name From Suppliers s JOIN Products p ON s.supplier_id =
```

```
p.product_id GROUP BY s.supplier_name HAVING count(DISTINCT p.category) > 1;
```

```
-- 6. Show each order along with the number of items it contains.
```

```
Select o.order_id, count(oi.order_item_id) As no_of_items From Orders o JOIN  
Order_Items oi ON o.order_id = oi.order_id GROUP BY o.order_id;
```

```
# b) Subqueries - Nested & Correlated:
```

```
-- 7. Find customers who have spent more than the average spending of all  
customers.
```

```
Select c.name From customers c JOIN Orders o ON c.customer_id = o.customer_id JOIN  
Order_Items oi ON o.order_id = oi.order_id GROUP BY c.customer_id HAVING  
sum(o.quantity * oi.price_each) > (select avg(total_spent) from (select  
sum(oi.quantity * oi.price_each) As total_spent From Orders o JOIN Order_Items oi ON  
o.order_id = oi.order_id GROUP BY o.customer_id) AS average_spending );
```

```
-- 8. List all products whose price is higher than the average product price in  
their category.
```

```
Select * From Products p WHERE price > (select avg(price) from Products Where  
category = p.category);
```

```
-- 9. Show customers who placed at least one order with total_amount greater than  
50,000.
```

```
Select c.name, sum(oi.quantity * oi.price_each) AS total_amount From customers c  
JOIN Orders o ON c.customer_id = o.customer_id JOIN Order_Items oi ON o.order_id =  
oi.order_id GROUP BY c.name, o.order_id HAVING sum(oi.quantity * oi.price_each) >  
50000;
```

```
-- 10. Find customers who placed more orders than the average number of orders per  
customer.
```

```
Select o.customer_id, c.name From Customers c JOIN Orders o ON c.customer_id =  
o.customer_id GROUP BY customer_id HAVING count(order_id) > ( select  
avg(orders_count) from (select count(order_id) As orders_count from orders Group By  
customer_id) AS avg_orders );
```

```
-- 11. Find the most expensive product(s) in the catalog.
```

```
Select product_name, price From Products WHERE price = (select max(price) from  
Products);
```

```
# c) Window Functions:
```

```
-- 12. Rank customers based on their total spending.
```

```
SELECT c.name, SUM(oi.quantity * oi.price_each) AS total_spent, RANK() OVER(ORDER BY  
Sum(oi.quantity * oi.price_each) Desc) As ranking FROM Customers c JOIN Orders o ON  
c.customer_id = o.customer_id JOIN Order_Items oi ON o.order_id = oi.order_id GROUP  
BY c.name;
```

```
-- 13. Find cumulative sales amount by order date.
```

```
SELECT o.order_date, SUM(oi.quantity * oi.price_each) AS daily_sales,  
SUM(SUM(oi.quantity * oi.price_each) ) OVER(ORDER BY o.order_date) AS
```

```
cumulative_sales FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY o.order_date;
```

```
-- 14.Get each customer's order count and show their percentage contribution.  
Select customer_id, count(order_id) As total_orders, count(order_id)*100 /  
sum(count(order_id) ) OVER() As percentage From Orders GROUP BY customer_id;
```

```
-- 15.Show the most recent order per customer.  
SELECT * FROM (Select *, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY  
order_date Desc) AS row_no From Orders) t WHERE row_no = 1;
```

```
-- 16.List each product with its sales quantity and rank products within each  
category by total sales.  
Select p.product_name, p.category, sum(oi.quantity) As total_sales, RANK()  
OVER(PARTITION BY p.category ORDER BY sum(oi.quantity) Desc) AS rank_in_category  
From Products p JOIN Order_Items oi ON p.product_id = oi.product_id GROUP BY  
p.product_name, p.category;
```

```
# d) CTE & Case:
```

```
-- 17.Categorize products as 'High', 'Medium', or 'Low' price using CASE.  
Select product_name, CASE WHEN price > 10000 THEN 'High' WHEN price BETWEEN 5000 and  
10000 THEN "Medium" ELSE "Low" END AS "price_category" From Products;
```

```
-- 18.Use a CTE to find top 3 customers by spending.  
WITH customer_spending AS (SELECT c.name, SUM(oi.quantity * oi.price_each) AS  
total_spent FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id JOIN  
Order_Items oi ON o.order_id = oi.order_id GROUP BY c.name) SELECT * FROM  
customer_spending ORDER BY total_spent DESC LIMIT 3;
```

```
-- 19.Use a CTE with CASE to classify customers by loyalty (based on number of  
orders).  
WITH order_counts AS (SELECT customer_id, COUNT(order_id) AS total_orders FROM  
Orders GROUP BY customer_id) SELECT customer_id, CASE WHEN total_orders >= 10 THEN  
'Gold' WHEN total_orders >= 5 THEN 'Silver' ELSE 'Bronze' END AS loyalty_status FROM  
order_counts;
```

```
-- 20.Find monthly revenue growth percentage compared to the previous month.  
WITH monthly_sales AS (SELECT DATE_FORMAT(o.order_date,'%Y-%m') AS month,  
SUM(oi.quantity * oi.price_each) AS revenue FROM Orders o JOIN Order_Items oi ON  
o.order_id = oi.order_id GROUP BY month) SELECT month, revenue, (revenue -  
LAG(revenue) OVER(ORDER BY month)) / LAG(revenue) OVER(ORDER BY month) * 100 AS  
growth_percentage FROM monthly_sales;
```

```
-- 21.Find top 2 customers per city based on total spending.  
SELECT * FROM (Select c.name, c.city, SUM(oi.quantity * oi.price_each)  
As total_spent, RANK() OVER(PARTITION BY c.city ORDER BY SUM(oi.quantity *  
oi.price_each) Desc) AS ranking From Customers c JOIN orders o ON c.customer_id =  
o.customer_id JOIN Order_Items oi ON o.order_id = oi.order_id GROUP BY c.name,
```

```
c.city) t WHERE ranking <= 2;
```

e) Miscellaneous Advanced Joins & Aggregations

```
-- 22.Find top 3 cities with highest sales revenue including shipper name.
```

```
Select c.city, sh.shipper_name, Sum(oi.quantity * oi.price_each) AS revenue From  
Customers c JOIN Orders o ON c.customer_id = o.customer_id JOIN Order_Items oi ON  
o.order_id = oi.order_id JOIN Shippers sh ON o.order_id = sh.shipper_id GROUP BY  
c.city, sh.shipper_name ORDER BY revenue LIMIT 3;
```

```
-- 23.List all orders with customer name, product name, supplier, and shipper.
```

```
Select o.order_id, c.name, p.product_name, s.supplier_name, sh.shipper_name From  
Orders o JOIN Customers c ON o.customer_id = c.customer_id JOIN Order_Items oi ON  
o.order_id = oi.order_id JOIN Products p ON oi.order_id = p.product_id JOIN  
Suppliers s ON p.product_id = s.supplier_id JOIN Shippers sh ON s.supplier_id =  
sh.shipper_id;
```

```
-- 24.Show total sales per supplier along with average order value.
```

```
SELECT s.supplier_name, SUM(oi.quantity * oi.price_each) AS total_sales,  
AVG(oi.quantity * oi.price_each) AS avg_order_value FROM Suppliers s JOIN Products p  
ON s.supplier_id = p.product_id JOIN Order_Items oi ON p.product_id = oi.product_id  
GROUP BY s.supplier_name;
```

```
-- 25.Show product categories that contributed more than 30% of total sales  
revenue.
```

```
SELECT p.category, SUM(oi.quantity * oi.price_each) AS revenue FROM Products p JOIN  
Order_Items oi ON p.product_id = oi.product_id GROUP BY p.category HAVING  
SUM(oi.quantity * oi.price_each) > (SELECT SUM(quantity * price_each) * 30/100 FROM  
Order_Items);
```